

Problem J1. Deliv-e-droid

Compute the score from the two input values: delivered packages give positive points, collisions give negative points, and an additional bonus is awarded when the number of packages is greater than the number of collisions. The formula is

$$50P - 10C + \begin{cases} 500, & P > C, \\ 0, & P \leq C. \end{cases}$$

Complexity. This is an $O(1)$ calculation.

Problem J2. Chili Peppers

Store the SHU value of each pepper name in a small lookup table. Then read the N pepper names, add the corresponding values, and print the total.

A sequence of `if/else` statements also works, but a dictionary or map mirrors the table in the statement and makes the implementation less error-prone.

Complexity. The running time is $O(N)$ and the memory usage is $O(1)$, since the table has fixed size.

Problem J3. Special Event

There are only five possible days. Maintain an array of five counters, initially all zero. For each person's availability string, increment the counters for all positions containing Y.

After all people are processed, find the maximum counter value and output every day whose counter equals this maximum. The only formatting detail is to print commas between the selected day numbers but not after the last one.

Complexity. The running time is $O(5N) = O(N)$, and the memory usage is $O(1)$.

Problem J4. Trianglane

Count the number B of black triangles. If no black triangles touched each other, the answer would be $3B$, because every black triangle has three sides. Whenever two black triangles share a side, that side is internal and should not be taped, so it subtracts 2 from the total.

Check horizontal adjacent pairs in both rows. Then check vertical adjacencies between the two rows; these can occur only at the columns whose triangle orientations allow the two triangles to share an edge. With zero-based columns, these are the even columns.

Complexity. A constant number of passes over the two rows gives $O(C)$ time and $O(C)$ or $O(1)$ extra memory, depending on how the rows are stored.

Problem J5. CCC Word Hunt

The intended method is systematic enumeration. Store the grid, then try every possible starting cell and every allowed first direction. For each choice, walk along the grid while comparing characters to the target word and stop as soon as the position leaves the grid or a mismatch occurs.

For the final subtask, the word may bend once at a right angle. A clean way to avoid a large amount of casework is to write a function that checks whether a word segment can be read from a position in a direction. Then enumerate the bend position and the two perpendicular directions.

Careful bounds checking is the main implementation challenge, especially near the edges of the grid.

Complexity. The number of start cells, directions, and possible bend positions is polynomial and small enough for the given grid sizes; a direct implementation runs in $O(RC |W| D)$, where D is the constant number of directions/cases considered.